What Limits Speculative Decoding at a Million Tokens? Greedy Loops, Rotary Stretching, and the Ring's Decode Step

Aman Kesarwani
Independent Researcher
amankesarwani@berkeley.edu

Abstract

Speculative decoding's short-context speedups do not survive million-token inference, and the reason is not the mechanism the technique's own literature would predict. We report three scientific findings and two systems measurements on 8×A100 80 GB SXM4 with FP4 weights and an NF4 KV cache. *First*, the acceptance such campaigns report is measured on degenerate text: under greedy decoding 9 of 10 rows in the released long-context first-window audit loop and 1 of 10 temperature-arm rows does, against 1 of 83 sampled rows at $T{=}0.7$. A greedy draft accepts a loop trivially, so a greedy acceptance number measures the text. *Second*, stretching the rotary basis of a model that ships the scaling dictionary costs a quarter of speculative acceptance for almost no perplexity: at 128 k, acceptance falls from 0.7272 to 0.5336 and 0.5006 while the paired perplexity ratio moves by $+0.05\%$ and $+1.56\%$. Acceptance is not a proxy for perplexity. *Third*, a natively 1M-trained target stays coherent at 1 M, within 9.5% of its own 128 k perplexity on the exactly aligned book. *Fourth*, our ring's decode bottleneck was the per-token key/value rotation: 256 rounds per token at a fixed $\sim$10.3 ms each regardless of payload, so per-round latency and not bandwidth. Removing it is worth 2.80 to $4.37\times$ decode-only with speculation and 3.30 to $5.32\times$ without. *Fifth*, speculation still pays 2.01 to $2.29\times$ on top, and the matched 1 M acceptance is unchanged (-0.1%). Against vLLM at bf16 with greedy decoding, which is not a precision-matched comparison, our prefill beats its entire run by 1.92 to $2.56\times$ while its decode stays 130 to $162\times$ faster per token than ours.

1 Introduction

Serving a million-token context on eight commodity GPUs forces two problems at once. The KV cache does not fit, which ring attention [Liu et al., 2023] solves by sharding the sequence dimension and rotating key/value blocks around the ranks. And every generated token still costs a full forward pass, which speculative decoding [Leviathan et al., 2023, Chen et al., 2023] attacks by having a small draft propose several tokens and verifying them with one target forward. Combining the two is the obvious systems move, and the interesting question is what happens to the second mechanism once the first is in place.

We built **RASD**, a ring-attention engine with an NF4 KV cache, chunked updates, an attention-sink prefix, rotary scaling where a target needs it, and speculative verification, and asked whether speculation still pays at 1 M. The measurements kept saying that something else was wrong, so the findings below lead with the science and close on our own engine.

40th Conference on Neural Information Processing Systems (NeurIPS 2026). Workshop: Workshop on Long Context Foundation Models (LCFM) @ NeurIPS 2026.

F1. Acceptance and perplexity dissociate under rotary stretching. On `meta-llama/Llama-3.1-8B`, a model that ships the `rope_scaling` dictionary the intervention edits, stretching the rotary basis at 128 k moves paired perplexity by $+0.05\%$ (factor 16) and $+1.56\%$ (factor 32) while speculative acceptance falls from 0.7272 to 0.5336 and 0.5006. The arms differ in nothing but `rope_factor` and the direction holds on five books of five. Position scaling is judged by perplexity; here it is nearly free by that measure and expensive by the measure a speculative decoder cares about.

F2. Greedy acceptance numbers at long context are measured on degenerate text. Under greedy decoding, 9 of 10 rows in the released long-context first-window audit degenerate into repetition, and 1 of 10 in the later temperature arms does, against 1 of 83 sampled rows at $T{=}0.7$; that one row is a rotary variant and §4.1 names it. A greedy acceptance number therefore measures the degeneracy and not the agreement between draft and target, which is why this campaign moved to sampling and why every row here carries an explicit periodicity verdict.

F3. A natively long-context target is coherent at 1 M. On the one exactly aligned book, the target's own perplexity at 1 M is 1.805 against 1.649 at 128 k, a $+9.5\%$ move; on the two books that share 91 to 95% of their scored tokens it is within 1.1%. Long context is therefore not intrinsically hostile to speculation, which is what made the two systems findings below worth chasing.

F4. Our ring's decode step was the key/value rotation. At 128 k each decode token needs 8 rotations over 32 layers, so 256 rounds of a `batch_isend_irecv` ring. A bare-process benchmark of that primitive, with no model attached, shows every payload costs the same $\sim$10.3 ms: 10.3124 ms for 4 KB against 10.3462 ms for 266 MB, a 66,000$\times$ payload range with a 0.4% spread. The cost is latency per round. Removing the rotation with decode-side context parallelism, which is prior work [Yang et al., 2024], is worth **2.80$\times$ / 3.77$\times$ / 4.37$\times$** decode-only with speculation and **3.30$\times$ / 4.26$\times$ / 5.32$\times$** without it at 128 k / 512 k / 1 M on matched (seed, book) cells, and the speedup grows with context, which is what a context-independent per-round cost predicts.

F5. Speculation still pays, and the 1 M acceptance is unchanged. With the rotation gone, speculation is worth $2.01\times$ / $2.29\times$ / $2.20\times$ decode-only over the same engine's target-only path at 128 k / 512 k / 1 M. Acceptance at 1 M under the fixed engine is -0.1% against the corrected old-engine baseline over 13 matched cells, and the target-only token stream from the fixed kernel is bit-identical to the ring's for twelve tokens before diverging where the baseline's own top-2 margin is 0.0000, so the speedups are not bought with a different model.

What this paper does not claim. The rotation-free decode kernel is prior work, cited for the technique and never claimed: Meta's context parallelism in its pass-KV and pass-Q forms [Yang et al., 2024] and vLLM's Decode Context Parallelism [Lee et al., 2026]. Our contribution is the diagnosis and the measurement. The comparison against vLLM is **not precision-matched** (§4.6).

2 Background and Related Work

Long-context attention. Ring attention [Liu et al., 2023] shards the sequence across ranks and pipelines exact attention through a ring of exchanges, so per-rank memory is independent of sequence length, with FlashAttention-2 [Dao, 2024] as the local kernel. Its published evaluation is a prefill story; what a decode step costs once every layer's key/value shard must visit every rank is what §4.4 measures.

Decode-side context parallelism (prior work). The decode-phase gap is closed in the literature, and we cite rather than claim it. Meta's context-parallel report [Yang et al., 2024] describes ring-context parallelism for million-token inference in pass-KV and pass-Q forms: instead of rotating every key/value block for each new query, a rank attends to the blocks it already holds and the partials are combined, with the query replicated in the pass-Q form. vLLM's Decode Context Parallelism [Lee et al., 2026] is the production implementation, motivated by KV memory that tensor parallelism cannot shard once the head count is small. We adopt the pass-Q form as *split-KV*.

Speculative decoding. Leviathan et al. [2023] and Chen et al. [2023] established draft-and-verify: a small model proposes γ tokens, the target verifies them in one pass, and the longest accepted prefix is kept with a residual resample for the rejected position. Acceptance α is the fraction the target accepts and the single-device speedup is roughly $(1-\alpha^{\gamma+1})/(1-\alpha)$; Cai et al. [2024] removed the separate draft model with multi-head prediction. All of it is evaluated at short context, and the interaction with a distributed KV cache is the setting here.

KV compression and position scaling. NF4 quantization [Dettmers et al., 2023] shrinks the cache and an attention-sink prefix [Xiao et al., 2024] stabilises it. Rotary embeddings [Su et al., 2024] are stretched past a model's training window by YaRN [Peng et al., 2023], an intervention normally validated by perplexity alone; §4.2 shows what that misses.

3 System and Setup

RASD in brief. RASD runs on one node of 8×A100 80 GB SXM4, with the target and a 1.3B draft on every rank. The KV cache is NF4 with chunked updates and a bf16 attention-sink prefix of the first 128 tokens on rank 0; the input is sharded across ranks; attention is ring attention with FlashAttention-2 in each local block; the weights are **FP4**, since NF4 is the KV codec, a distinction the reviewed version blurred. At 1 M the system peaks at **41.2 GiB allocated per rank** over the ranks with 69 GiB reserved, where the vanilla HuggingFace stack runs out of memory at 128 k single-rank. Rotary scaling is applied at a fixed factor 8 at 8 k and above and not at all below that, anchored on each context's own length because the `transformers` version in use ignores `original_max_position_embeddings`.

Targets. Two appear. The long-context campaign runs `gradientai/Llama-3-8B-Instruct-Gradient-1048k`, trained to a 1 M window, so nothing is stretched; the rotary intervention of §4.2 runs `meta-llama/Llama-3.1-8B`, which ships the scaling dictionary under test, with a tokenizer-sharing draft. A ceiling test on `meta-llama/Llama-2-7b-hf`, the 4 k-window model the engine was first built on, is a capability reference only and no ratio here uses it; it is detailed in Appendix E.

Protocol of every reported result. Prompts are PG-19 narrative [Rae et al., 2020], five books, windowed so each rung's scored continuation ends at the same document position as the 1 M rung, with the alignment verified from `continuation_sha256` and the recorded offsets rather than from a pool name. Decoding is $T{=}0.7$ sampling, 128 new tokens, $\gamma{=}4$ speculative steps, draft window cap 4096, seed 42 unless stated, and every row carries a repetition verdict from the detector of §4.1. Decode-only means $(t_{\text{row}} - t_{\text{ttft}})/(\text{tokens}-1)$, so prefill is excluded and a decode-path change is not credited with prefill it did not touch. Every row also records the engine that produced it (`ring_decode_mode`, patched-layer count, git commit), so a row that cannot prove its engine is identifiable and void.

The draft saw the prompt's first 4096 tokens. In every run here whose prompt exceeded 4096 tokens, and every run in this paper does, the draft was conditioned on the prompt's *first* 4096 tokens rather than on the recent window. The reviewed version described the opposite. We disclose it rather than argue it: it lowers the *level* of acceptance on every long-context row, and it does not affect any *contrast* between arms that share the conditioning, which is every contrast reported here.

4 Results

4.1 What a greedy acceptance number is measuring

The campaign behind this paper began under greedy decoding, and its acceptance numbers were high in a way that deserved suspicion: a draft that proposes the token the target would emit anyway is accepted, and a target that has fallen into a loop is maximally predictable. We measured the degeneracy instead of assuming it. The detector is the project's absolute repetition rule: a verbatim repeat span of 128 tokens or more, or 60% of tokens inside a repeated 16-gram, or a periodic tail covering 50% of the row.

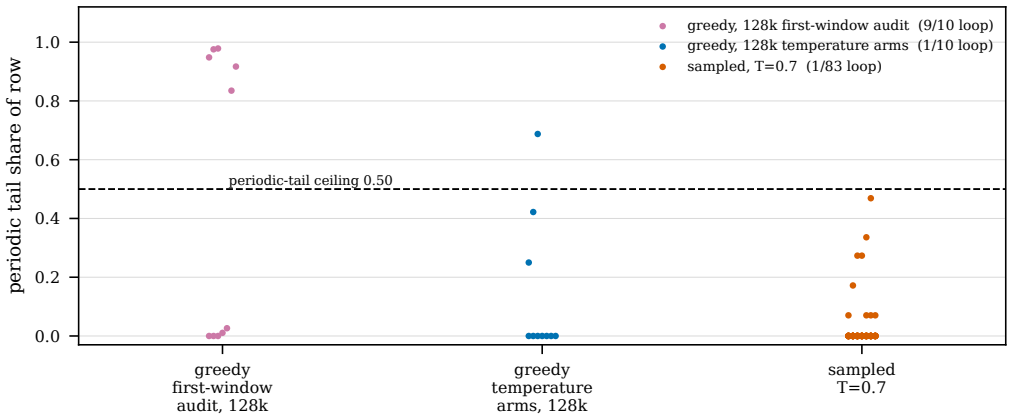


Figure 1: Periodic tail share per row for the rows that ship with a per-row verdict, against the 0.50 ceiling. Left and middle: greedy rows from two campaigns, 9 of 10 looping in the first-window audit and 1 of 10 in the temperature arms. Right: every sampled row in the release, old and fixed engines together, 1 of 83 looping. The long-context census that motivated the move to sampling (9 of 15) is in Appendix A; its per-row sidecars are not part of this release, so it is quoted there and not used here.

Table 1: The rotary intervention at 128 k on `meta-llama/Llama-3.1-8B`: five books per arm, seed 42, FP4 weights with an NF4 KV cache. The perplexity ratio is the paired measurement against the native arm on the same continuation. The `tok/s` column is **old-engine** throughput, reported for completeness only: the finding is the acceptance and perplexity contrast, and no speedup is claimed from these arms.

Arm	n	Acceptance	ppl ratio (vs native)	tok/s *[old engine]*
native	5	0.7272	1.0000	0.9109
factor 16	5	0.5336	1.0005	0.7699
factor 32	5	0.5006	1.0156	0.7473

Figure 1 is the census over the rows that ship with a per-row verdict. Under greedy, **9 of 10** rows in the long-context first-window audit loop and **1 of 10** in the later 128 k temperature arms does. Under $T{=}0.7$ sampling, **1 of 83** rows does, and it is not a headline row: it is `ROPE_EXT_SAMP_F16_SPEC_pg19_train_2768_s42`, a rotary-variant arm on book 2768 whose share of tokens inside a repeated 16-gram is 77% with no periodic tail. It is outside the headline pools and we name it rather than average it away.

Three consequences are applied in this paper rather than merely noted. The headline protocol is sampled at $T{=}0.7$, with the earlier greedy numbers reported separately and labelled. A looping row is named wherever one appears, never smoothed into a mean. And the temperature ablation points *up*: at 128 k, $T{=}1.0$ reaches acceptance 0.7369 against 0.4681 at the headline $T{=}0.7$, with 0 of 5 rows degenerate in both arms.

4.2 Rotary stretching: acceptance and perplexity dissociate

The natural assumption is that an intervention which costs little in perplexity will cost little in acceptance, because the draft and the target see the same positions. We tested it on `meta-llama/Llama-3.1-8B` at 128 k, with five books per arm and the arms differing in nothing but `rope_factor`.

Acceptance falls from **0.7272** (native) to **0.5336** (factor 16) and **0.5006** (factor 32), a **27** to **31%** relative loss, while the paired perplexity ratio moves by $+0.05\%$ and $+1.56\%$. The direction holds on five books of five, so it is not a book effect, and because the arms differ in nothing else the drop is a **rotary effect at $n{=}3$ arms**. The two stretched factors land far closer to each other (6.6% apart) than either lands to native, which is what a saturating intervention looks like; at $n{=}3$ books

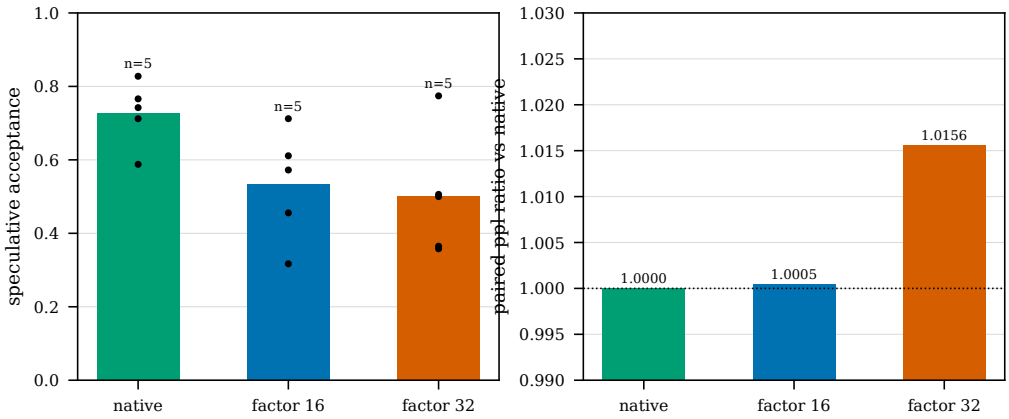


Figure 2: Left: speculative acceptance per arm, with the five books drawn as points so the direction can be checked per book rather than only in the mean. Right: the paired perplexity ratio against the native arm. Stretching the rotary basis costs 0.05% and 1.56% of perplexity and 27% and 31% of acceptance.

their difference was 0.0012 and was read as exact saturation, and at $n{=}5$ it is 0.033, so that specific reading is withdrawn and the weaker claim stands.

A speculative decoder is only as good as the agreement between the draft's proposal distribution and the target's. Position scaling is normally evaluated by perplexity, and by that measure it is nearly free here. The same intervention costs almost a third of the acceptance, and therefore almost a third of the speculation speedup, and nothing in the perplexity number reveals it. Acceptance under any change to the target's positional treatment has to be measured. One scope note: every arm here ran on the old engine, so the absolute levels sit on the pre-fix decode path; the contrast is unaffected, because the fixed kernel does not change which tokens are generated (§4.5).

4.3 Coherence at 1 M on a natively long-context target

The perplexity comparison above cannot reach 1 M, so we check the long-context property directly on a target trained to 1 M (`gradientai/Llama-3-8B-Instruct-Gradient-1048k`), where nothing is being stretched and the question is only whether the model is still a model. The rungs are matched by `continuation_sha256` and the recorded window offsets, never by a pool name.

On the **one exactly aligned book** the target's perplexity is **1.805 at 1 M** against **1.649 at 128 k**, a $+9.5\%$ move. On the two books that share 91 to 95% of their scored tokens the difference is within **1.1%**. So a natively long-context target does stay coherent across an eightfold context increase, which is the property the whole campaign depends on, but the strongest form of that claim rests on one exactly aligned book and we say so rather than pooling three and reporting a tighter number. The measurement is prefill-only, so the decode fix cannot move it: the numbers are identical under both engines.

4.4 The ring's key/value rotation was the decode step

At 128 k, each decode token requires 8 rotations over 32 layers, so **256 rounds** of a `batch_isend_irecv` ring per token. Figure 3 measures that primitive with no model attached, over five payloads from 4 KB to 266 MB, the latter being the key and value of a 128 k window. Every payload costs the same $\sim$10.3 ms: the range spans 10.3124 to 10.3462 ms, a 66,000$\times$ payload range with a **0.4% spread** and a worst rank-to-rank spread of 0.056 ms over eight ranks. A round is latency, not bandwidth, and it is paid once per round however small the round is. On the same box the profiled ring path spent about 9.3 ms per round, so the bare primitive accounts for the whole cost of the decode step and leaves nothing to attribute to per-round engine overhead or rank skew.

Removing the rotation is prior work (§2), adopted here as **split-KV**: each rank computes attention for the replicated query against only its own prefill shard, producing a partial output and a log-sum-exp, and the partials are merged by one MAX and one SUM all-reduce; for a single query token the

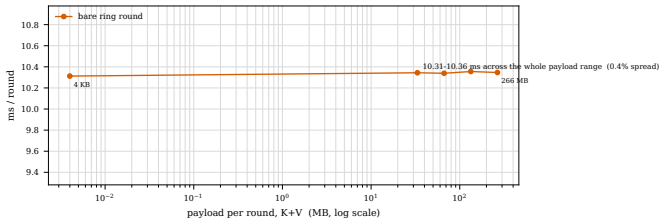


Figure 3: A bare-process benchmark of the ring primitive the engine uses (`batch_isend_irecv`, 8 ranks, 256 rounds per payload, no model). The line is flat: a round costs a fixed $\sim$10.3 ms from 4 KB to 266 MB, so the ring's decode cost is per-round latency and cannot be amortised by moving more data per round. NCCL selected `P2P/CUMEM/read` on a full NV12 mesh.

Table 2: Decode time per token, old ring engine against split-KV, on *matched* (seed, book) cells: one value per cell on both sides, plumbing repeats dropped, and the two seed stages hit by the configuration bug of §4.5 excluded. Speedup is the ratio of decode-only times.

Context	Arm	Decode s/token		Speedup	n
		old ring	split-KV		(cells)
128k	spec	1.4779	**0.5275**	2.80×	3
128k	target-only	3.4925	**1.0596**	3.30×	1
512k	spec	3.5344	**0.9384**	3.77×	3
512k	target-only	9.9900	**2.3460**	4.26×	1
1M	spec	6.0506	**1.3830**	4.38×	13
1M	target-only	17.0902	**3.2107**	5.32×	2

partials are tiny and the replicated tail is counted exactly once. Our contribution is the diagnosis and the measurement, not the technique.

Table 2 and Figure 4 are the headline. Decode-only, the fix is worth **2.80× / 3.77× / 4.37×** with speculation and **3.30× / 4.26× / 5.32×** without it at 128 k / 512 k / 1 M, and **the speedup grows with context**, which is the mechanistic claim rather than a by-product: as the rest of the decode step grows with the context, the rotation's fixed share of it shrinks relative to the rotation-free path that replaces it, so the ratio improves. Two caveats we state rather than bury. The matched old-against-new comparison is one cell at 512 k and two at 1 M, because that is how many books the old engine measured there, while the fixed engine's own target-only arms were widened from one and two books to three and five by a later top-up run. And the four diagnostic arms that isolated the mechanism, `kv_block_size` 32768 (1.24×) and `repeat_kv` after the rotation (+2.7%), are bit-identical in token stream and inert under split-KV, so they carry no claim here; they are reported because they ruled out the cheaper explanations first.

4.5 Speculation still pays, and the baseline is corrected

With the rotation gone, the question the paper started from can be asked on an engine whose decode step is no longer dominated by a collective. The answer is yes: **2.01× / 2.29× / 2.20×** decode-only at 128 k / 512 k / 1 M over the same engine's own target-only path, with three books against one in the 128 k arms, three against three at 512 k, and five against five at 1 M. End-to-end, where prefill is included, the same comparison reads 1.95× / 1.76× / 1.41×, because at 1 M a row is 570 s of which about 340 s is prefill: with the decode step fixed, prefill is the long pole and any statement about 1 M throughput has to say which of the two it means.

Two of those denominators used to be one and two books. We topped them up after the confirmatory run, because a ratio whose denominator is one row is a ratio of one row to another. The 512 k target-only mean is 2.148 s/token over three books against the 2.346 single-book value it replaces, and the 1 M mean is 3.155 over five against 3.211 over two. Both moves are downward, so the quoted ratios are lower than the thin ones: the thin denominator was flattering, by 8% at 512 k. The book-to-book spread inside the fixed engine is 1.27× at 512 k and 1.30× at 1 M, wider than the 1.04× spread across seeds, so books and seeds are not interchangeable units.

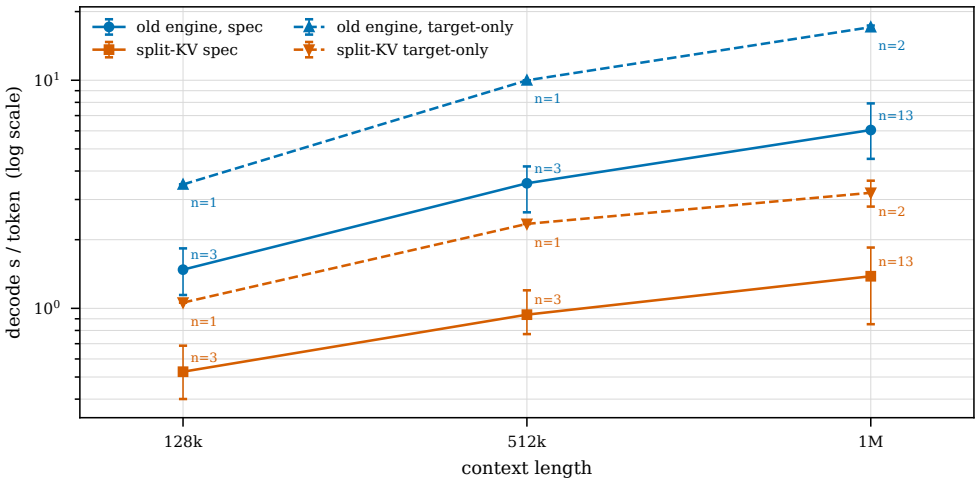


Figure 4: Decode time per token against context length, both engines and both arms, on the same matched cells. Whiskers are the range over cells and n is the number of cells both engines measured. The gap widens with context, which is the signature of a context-independent per-round cost being removed from a step whose other terms grow.

Table 3: vLLM (bf16, bf16 KV, greedy) against the ring engine and the split-KV engine (FP4 weights, NF4 KV, $T{=}0.7$ sampled) on the same prompts. Prefill is the first-token time and decode is per token. **Not a precision-matched comparison**: the systems differ in precision and in sampling.

Context	Prefill (s)		Decode (s/token)		
	vLLM	RASD	vLLM	old ring	split-KV
128k	16.85	8.76	0.0081	3.49	1.06
512k	222.80	88.56	0.0145	9.99	2.35
1M	859.60	335.34	0.0234	17.09	3.21

The corrected old-engine baseline. Two stages of the seed ablation, the ones meant to run seeds 123 and 456 at 1 M, ran seed 42 instead, because the config builder merges a level into the defaults and then sets the seed from the *defaults'* list, so a level that declares its own `seeds` has it recorded and overwritten. Rebuilt from rows that were never affected, on 13 matched (seed, book) cells, the 1 M speedup is **4.375**$\times$ rather than $4.033\times$ and the 1 M acceptance difference is -0.1% rather than -10.4%: the fixed engine's acceptance is *unchanged*, and the previously reported drop was the bug. The old engine's own seed spread is 23.0% of the mean on the matched books, so per-seed movement of -15.7%, $+10.9\%$ and $+11.3\%$ sits inside the old engine's own seed-to-seed variation, in both directions.

The speedups are not bought with a different model. An exactly equivalent kernel can still move an acceptance number, because the accept/reject decision compares the target's distribution against the draft's proposal, so one flipped near-tie changes the emitted prefix and every later proposal with it. We checked the kernel directly: the target-only stream under split-KV is **bit-identical to the ring baseline for twelve consecutive tokens** and diverges at a step where the baseline's own top-2 logit margin is **0.0000**, against a row median of 2.9375, with prefill perplexity identical to six decimals.

4.6 Comparison against a production engine

We benchmark vLLM 0.6.3 at tensor-parallel 8 on the same prompts, fed the same token ids from our own sidecars, so the units match. The comparison is **not precision-matched**: vLLM runs **bf16 weights with a bf16 KV cache and greedy decoding**, while RASD runs **FP4 weights with an NF4 KV cache** at $T{=}0.7$ with speculation. Every ratio below carries that mismatch.

Two readings that point in opposite directions, and both belong here. **Our prefill beats vLLM's entire run**, meaning its prefill plus its 128 decode steps, by $1.92\times$ / $2.52\times$ / $2.56\times$ at 128 k / 512 k / 1 M; that is a property of the ring prefill, untouched by the decode fix, and it is the strongest end-to-end statement this paper can make against a production engine. And **vLLM's decode is 130 to 162$\times$ faster per token than ours even after the fix**, against 429 to 731$\times$ before it: removing the rotation eliminated roughly 70% of the decode deficit and left our decode far behind a production engine. Speculation does not close the gap either, since our spec decode is $65\times$, $65\times$ and $59\times$ vLLM's. The honest framing is that the ring's rotation was the decode *pathology*, and removing it made our decode merely slow.

5 Limitations

- **Sample size.** The matched comparison is 3 spec and 1 target-only cell at 128 k, 3 and 1 at 512 k, 13 and 2 at 1 M. Inside the fixed engine the target-only arms are 1, 3 and 5 books. The five units in a five-book arm are books, not seeds, and the book spread is wider than the seed spread.
- **The draft saw the prompt's first 4096 tokens** (§3), so every acceptance level here is a lower bound; contrasts between arms are unaffected.
- **One node, one fabric, one precision.** Everything is 8×A100 80 GB SXM4 over NVLink. Multi-node moves the collective to a slower fabric, which is exactly the regime where per-round latency, the quantity §4.4 measures, grows. There is no bf16 row: every number carries the FP4/NF4 precision cost.
- **Acceptance under split-KV is explained, not tested.** The 1 M acceptance is unchanged over 13 matched cells while per seed it moves both ways inside a 23.0% old-engine spread. The mechanism we offer, one flipped near-tie changing the emitted prefix, is consistent with every measurement and was not tested directly.
- **Prefill is now the 1 M pole**: a 1 M row is 570 s of which about 340 s is prefill, so the end-to-end 1 M speedup is $1.41\times$ against $2.20\times$ decode-only.
- **An unattributed elementwise term.** In the profiled 128 k decode window, 19.3% of device time was elementwise kernels never attributed to an operation. With the rotation gone that is a larger share of what remains, and it is the obvious next optimisation.
- **Two withdrawn claims are recorded rather than deleted** (Appendix A): the reviewed version's "communication is at most 1.2% of wall time", whose bucket recorded 0.31 to 0.37$\times$ of the physical floor for the same runs, and the $4.4\times$ "native context" speedup from one pair in one session.

6 Conclusion

We asked what limits speculative decoding at a million tokens, and the answer arrived in three parts, none of them about speculation itself. Under greedy decoding the acceptance numbers this kind of campaign reports are measured on degenerate text: 9 of 10 long-context rows in the released first-window audit loop, and 1 of 83 sampled rows does. Once the protocol is sampled at $T{=}0.7$, stretching a model's rotary basis costs a quarter of the acceptance (27 to 31% relative) for 0.05 to 1.56% of perplexity, so acceptance and perplexity dissociate and the second cannot stand in for the first. A natively 1M-trained target stays coherent at 1 M, within 9.5% of its own 128 k perplexity on the exactly aligned book, so the regime is not intrinsically hostile to speculation.

What was hostile was our engine. Rotating each layer's key/value shard around eight ranks costs 256 rounds per decode token, and a bare-process benchmark shows each round costs a fixed $\sim$10.3 ms whatever the payload, from 4 KB to 266 MB within 0.4%: the decode step was latency per round. Adopting decode-side context parallelism, which is prior work, removes those rounds and is worth 2.80 to $4.37\times$ decode-only with speculation and 3.30 to $5.32\times$ without it, growing with context as a context-independent per-round cost predicts, while speculation still pays 2.01 to $2.29\times$ on top and the matched 1 M acceptance is unchanged at -0.1%. Against vLLM at bf16 with greedy decoding, which is not a precision-matched comparison, our prefill beats its entire run by 1.92 to $2.56\times$ and its decode remains 130 to $162\times$ faster per token than ours. The rotation was the decode pathology; removing it made our decode merely slow. That is the honest systems claim, and it is the one this paper makes.

References

Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D Lee, Deming Chen, and Tri Dao. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. *arXiv preprint arXiv:2401.10774*, 2024.

Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023.

Tri Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. In *International Conference on Learning Representations (ICLR)*, 2024.

Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. *arXiv preprint arXiv:2305.14314*, 2023.

Seonghee Lee, Sungsoo Ha, Omri Almog, and Lucas Wilkinson. Efficient decode context parallelism with vLLM for long context workloads. vLLM Blog, August 2026. URL `https://blog.vllm.ai/2026/08/07/decode-context-parallelism.html`. Accessed 2026-10-13.

Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning (ICML)*, 2023.

Hao Liu, Matei Zaharia, and Pieter Abbeel. Ring attention with blockwise transformers for near-infinite context. *arXiv preprint arXiv:2310.01889*, 2023.

Bowen Peng, Jeffrey Quesnelle, Honglu Fan, and Enrico Shippole. YaRN: Efficient context window extension of large language models. *arXiv preprint arXiv:2309.00071*, 2023.

Jack W Rae, Anna Potapenko, Siddhant M Jayakumar, Chloe Hillier, and Timothy P Lillicrap. Compressive transformers for long-range sequence modelling. In *International Conference on Learning Representations (ICLR)*, 2020.

Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 2024.

Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. *International Conference on Learning Representations (ICLR)*, 2024.

Amy Yang, Jingyi Yang, Aya Ibrahim, Xinfeng Xie, Bangsheng Tang, Grigory Sizov, Jeremy Reizenstein, Jongsoo Park, and Jianyu Huang. Context parallelism for scalable million-token inference. *arXiv preprint arXiv:2411.01783*, 2024.

A Changes from the reviewed version

Every claim in the reviewed submission, with its status here. Corrections cite the audit item that found them (`results/mlsys/LCFM_RECONCILE.md`, AUDIT E1 to E23).

B The corrected baseline and the matched-cell tables

The configuration bug. `run_experiment.py:590-612` merges a level into the defaults and then sets the run's seed from the *defaults'* `seeds` list, so a level that declares its own `seeds` has it recorded and then overwritten. Exactly two stages did: `SA_SEED_123_1M_SPEC` and `SA_SEED_456_1M_SPEC`, both of which ran seed 42 while their run ids said otherwise. Every old-engine pool containing them inherited the error. The corrected baseline is built from rows that were never affected, seed 42 from the sampling session and seeds 123 and 456 from a dedicated rerun; the exclusion list is printed by `scripts/mlsys_seed_baseline_audit.py` and reproduced in `results/mlsys/SEED_BASELINE_AUDIT.txt`.

Table 4: Status of every claim in the reviewed version. KEPT means it stands as written; CORRECTED gives the old reading and the new one; REMOVED means the claim is withdrawn and not replaced. Audit items are E1 to E23 of `results/mlsys/LCFM_RECONCILE.md`.

Claim in the reviewed version	Status	Reason
Communication at most 1.2% of wall (E1)	REMOVED	The bucket recorded 0.31 to $0.37\times$ of the physical wire floor, so it never saw the transfers.
Idle at or above 50% is collective wait (E2)	REMOVED	Measured outside the region the collectives run in.
The tick-gate barrier (E3)	REMOVED	Not the code that produced the reported numbers.
The rotation is hidden and the draft pass is free (E4)	CORRECTED	The rotation is issued before attention, and the claimed draft overlap was never implemented.
Positioning table and novelty framing (E5)	CORRECTED	Split-KV is prior work, and the decode-overlap cell was not implemented.
The draft saw the last 4096 tokens (E6)	CORRECTED	It saw the first 4096, on every run whose prompt exceeded that (`d89cf41`).
Anchor is the model's original window (E7)	CORRECTED	Each context was anchored on its own length.
Factor $\lceil N/4096 \rceil$ (E8, E9)	CORRECTED	A fixed factor 8 at 8 k and above, and none below.
Verify-loop defects (E10)	CORRECTED	Per AUDIT E10.
Three independent seed samples (E11)	CORRECTED	All three seeds ran one prompt.
95% intervals at n=3 (E12)	CORRECTED	Ranges are reported, with the n.
Nominal 1 M context (E13)	CORRECTED	1,047,551 prompt tokens here against 1,002,984 in the reviewed build.
Speedup regresses to $0.94\times$ and $0.97\times$ (E14)	REMOVED	Decode tokens per round are flat, and the same-code rerun does not reproduce it.
A control explains the bimodality (E15)	REMOVED	The control differs from the long-context cells in five ways at once.
$4.4\times$ at native context (E16)	REMOVED	One pair in one session, above what its own acceptance can deliver.
The speedup formula (E17)	CORRECTED	Applied to the wrong quantity; throughput is tokens over time.
Peak memory per rank (E18)	CORRECTED	The peak was rank 0 only; over ranks it is 41.2 GiB allocated.
"NF4 weights" (E19)	CORRECTED	The weights are FP4; NF4 is the KV codec.
NF4 KV description (E20)	CORRECTED	Per AUDIT E20.
Design-point sweeps (E21)	CORRECTED	Chunk size does not bound memory, depth 2 repeats depth 1, and the γ optimum is unsupported.
Prompts and citations (E22)	CORRECTED	The synthetic prompt is not the prior-work protocol, and one citation does not exist.
Qualitative comparison (E23)	KEPT	Stands, with the audit's caveat.
Greedy census, 9 of 15 rows looping	KEPT, MOVED HERE	Its per-row sidecars are **not released**, so the main text quotes released numbers only.
Profiler time-breakdown table and figure	CUT	The comm bucket in it is below the physical floor, so it bounds nothing.
Llama-2 quantization and rotary perplexity table	CUT	Labelled by intent rather than configuration, and one claim cited a number that does not exist.
RULER infrastructure appendix	CUT	Infrastructure without a run grid supports no claim.

Effect. The 512 k target-only mean moves from 2.3460 over one book to 2.1480 over three, and the 1 M mean from 3.2107 over two books to 3.1553 over five. The spec-over-target ratios in §4.5 are computed on these wider arms; the matched speedups in Table 5 are not affected, because the new books have no old-engine counterpart. Both movements are downward, so widening the denominator lowered the reported ratios rather than raising them.

C The decode diagnosis in detail

The five payloads span a factor of 66,000 in bytes and 0.4% in time, so the round is dominated by fixed latency and not by transfer. This is why the fix is to remove the rounds rather than to move more data per round.

Correctness. The target-only stream under split-KV is bit-identical to the ring baseline for twelve consecutive tokens and then diverges at a step where the baseline's own top-2 logit margin is **0.0000**,

Table 5: Decode time per token on matched (seed, book) cells, one value per cell on both sides, plumbing repeats dropped, bug-affected seed stages excluded. This is the table the main text quotes.

Context	Arm	Decode s/token		Speedup	n
		old ring	split-KV		(cells)
128k	spec	1.4779	**0.5275**	2.80×	3
128k	target-only	3.4925	**1.0596**	3.30×	1
512k	spec	3.5344	**0.9384**	3.77×	3
512k	target-only	9.9900	**2.3460**	4.26×	1
1M	spec	6.0506	**1.3830**	4.38×	13
1M	target-only	17.0902	**3.2107**	5.32×	2

Table 6: The corrected old-engine baseline at 1 M, over the 13 matched (seed, book) cells where both engines have a row. The old engine's spread across its own seeds is 23.0% of the mean on these books.

seed	old acceptance	fixed-engine acceptance	change
42	0.5246	0.4424	−15.7%
123	0.4656	0.5164	+10.9%
456	0.4466	0.4969	+11.3%
pooled over 13 cells	0.4839	0.4834	−0.1%

against a row median of 2.9375 and a minimum of 0.0000: an exact tie broken the other way. Greedy repeats the pattern, and the speculative row diverges at index 1 with margin 0.8750 against a row median of 0.9375, below the row's own median. Prefill perplexity is identical to six decimals under both configurations, and the perplexity gate is in any case blind to a decode-path change, because it scores a prefill forward with `use_cache=False`.

D Ablations kept from the reviewed version

The design-point sweeps are **old-engine** results on the **synthetic repeated paragraph** at 64 k on 8×A100 40 GB, three seeds on one prompt, so the bracketed figures are ranges over those three runs and not confidence intervals. They are kept because they document how the system's knobs were chosen, and because two of the reviewed version's readings of them were wrong: the chunk size is a transmission chunk and does not bound memory, and prefetch depth 2 executes the same code path as depth 1, so that axis had two arms and not three. The block-size effect is specific to the ring decode path and is inert under split-KV.

E Reproduction and evidence tree

Where every number lives. Nothing in this paper is transcribed by hand. The figures and tables are written by `scripts/lcfm_figures.py`, which reads the CSVs below and writes `results/lcfm/figs/` and `results/lcfm/tables/`; those tables are *included* by this document rather than copied into it.

- **Rotary intervention** `results/mlsys/sampling_session/session_a_ROPE_SAMP_*_SPEC.csv` and `results/mlsys/n5_extension/session_a_ROPE_EXT_*`, with the paired perplexity ratios in `session_a_rope_fix_gate.csv`.
- **Degeneracy** the per-stage `repetition_*.csv`, produced by `scripts/mlsys_repetition_report.py` from the token sidecars with the ceilings in `src/analysis/repetition.py`; where no such CSV exists the row is scored from the sidecar by the same module.
- **Coherence** `QUALITY_FIX_{128K,1M}_*` with the alignment table `results/mlsys/sampling_session/alignment.csv`.
- **The decode fix** the confirmatory run in `results/mlsys/rerun_r1/` and `rerun_r2/` (31 ok rows, each recording its engine and commit), the diagnostic arms in `results/mlsys/decode_df/arms/df_arms.csv`, and the top-up in `results/mlsys/rerun_topup/`.

Table 7: The target-only top-up. The fixed engine's own target-only arms were widened after the confirmatory run; the matched old-against-new cells are unchanged by it.

stage	book	context	decode s/token	provenance
TP_GATE_128K_TARGET	train_0	128 k	1.051	split_kv, 32 layers
TP_1M_TARGET	train_915	1 M	2.826	split_kv, 32 layers
TP_1M_TARGET	train_2204	1 M	3.126	split_kv, 32 layers
TP_1M_TARGET	train_2768	1 M	3.402	split_kv, 32 layers
TP_512K_TARGET	train_1	512 k	1.847	split_kv, 32 layers
TP_512K_TARGET	train_915	512 k	2.251	split_kv, 32 layers

Table 8: Bare-process ring benchmark: the batch_isend_irecv ring primitive the engine uses, 8 ranks, 256 rounds per payload, no model attached. Reported for rank 0; the worst rank-to-rank spread over all eight ranks is 0.056 ms.

payload	size	K+V bytes	ms/round	GB/s
tiny_4kb	4 KB	4,096	**10.3124**	0.0
mb33	33.3 MB	34,918,400	10.3437	3.22
k8_66mb	66.6 MB	69,836,800	10.3389	6.44
mb133	133.2 MB	139,673,600	10.3557	12.86
mb266	266.3 MB	279,347,200	**10.3462**	25.74

- **The bare ring** results/mlsys/decode_df/ring_bench/ringbench.csv, from scripts/mlsys_ring_microbench.py.
- **The corrected baseline** scripts/mlsys_seed_baseline_audit.py, which prints the exclusion list and the matched-cell table.
- **vLLM reference** results/mlsys/vllm_run2/vllm_baseline.csv (bf16, bf16 KV, greedy, unit-matched on every row). The earlier run at results/mlsys/vllm_run/ is retained but its first-token and decode clocks are absolute monotonic readings and are unusable; only its end-to-end numbers agree, within 2.4% at 128 k and 0.1% elsewhere.

The ceiling test, in one place. Vanilla HuggingFace generate() with FlashAttention-2, bf16 weights and a bf16 KV cache, single rank, no quantization and no sequence parallelism, on meta-llama/Llama-2-7b-hf: 32 k completes at 30.1 GiB and 7.25 tok/s, and every context of 128 k and above runs out of memory during prefill. The model differs from the targets used for the throughput results (§3), so this is a capability reference for the standard stack and nothing more; no ratio in this paper uses it.

Building this paper. cd manuscript/lcfm and compile main.tex with latexmk or tectonic; the bibliography is references.bib. The long version of this work, including the withdrawn material and the full Ablations, lives in manuscript/arxiv/.

Table 9: The engine arms: one model load, four configurations, 128 k target-only at 16 tokens each. The two ring variants are bit-identical in token stream to `ring2048` and the split-KV kernel is bit-identical to `ring32768` for twelve tokens.

configuration	target-only s/token	vs `ring2048`	vs `ring32768`	token stream
`ring2048`	3.5637	1.000×		baseline
`ring32768`	2.8749	1.240×	1.000×	bit-identical
`ring32768x`	2.7985	1.273×	1.027×	bit-identical
`splitkv`	**1.0011**	**3.560**×	**2.872**×	12 tokens, then a tie

Table 10: M3 ablation results (64 k context, 8×A100-SXM4-40 GB). Mean and range over 3 seeds: **three seeds on one synthetic prompt**, not three independent samples, so the bracketed intervals are ranges rather than confidence intervals (AUDIT E11, E12); horizontal rules separate the five sweep axes. Deterministic early-EOS rows with tokens_generated < 20 are excluded; see §Error Analysis.

Axis	Level	Throughput (tok/s) [range]	Accept. [range]	n
A1: Draft model	TinyLlama-1.1B	0.84 [0.82, 0.85]	0.235 [0.226, 0.252]	3
	Sheared-1.3B	0.87 [0.84, 0.90]	0.253 [0.231, 0.277]	3
A2: Spec steps (k)	k=2	0.81 [0.74, 0.85]	0.381 [0.303, 0.424]	3
	k=4	0.86 [0.81, 0.91]	0.253 [0.231, 0.277]	3
	k=6	0.84 [0.80, 0.88]	0.176 [0.149, 0.193]	3
	k=8	0.86 [0.80, 0.89]	0.147 [0.124, 0.160]	3
	k=12	0.85 [0.82, 0.88]	0.111 [0.105, 0.119]	3
A3: KV block size	256	0.62 [0.58, 0.65]	0.253 [0.231, 0.277]	3
	512	0.87 [0.81, 0.93]	0.253 [0.231, 0.277]	3
	1024	1.11 [1.06, 1.15]	0.253 [0.231, 0.277]	3
	2048	1.25 [1.20, 1.30]	0.253 [0.231, 0.277]	3
A4: Prefetch depth	sync	0.88 [0.84, 0.90]	0.253 [0.231, 0.277]	3
	async-1	0.87 [0.81, 0.92]	0.253 [0.231, 0.277]	3
	async-2	0.86 [0.84, 0.89]	0.253 [0.231, 0.277]	3
A5: Target model	Llama-2-7B	0.86 [0.83, 0.90]	0.253 [0.231, 0.277]	3